
# LLM Efficiency Techniques

**AUTHORS**

Titouan Guerin & Yacine Chettab

January 9, 2026

# 1 Introduction

Despite achieving human like results on numerous NLP tasks, LLMs are expensive to train and use in real time due to their size and computing requirements. To make LLMs actually usable in the real world, many methods exist to make them more efficient, such as Low-rank adaptation (LoRA), INT8 quantization and PEFT which provide workable ways to lower memory use, training time, and inference cost without sacrificing accuracy.

In this practical, we experimented with HuggingFace's PEFT library to examine various efficiency strategies, coded LoRA from scratch, and applied INT8 quantization to feedforward layers. The trade-offs between base models and the added effeciency methods were explored by benchmarking all models on memory, training time, inference speed, and test accuracy.

# 2 Benchmarking Methodology

In this study, we use **DistilBERT**, a smaller version of BERT that keeps most of the original model's performance while reducing its size. Our task is binary sentiment classification on the IMDB dataset, predicting whether a given movie review is positive or negative. The dataset is composed of $9500$ examples for the train split, and $500$ for the validation split.

As a baseline, the original DistilBERT model achieves an accuracy of **89.60%** on the evaluation set, which will serve as a reference for all the following experiments.

We explored several parameter and computation efficiency techniques to adapt the model:

- **LoRA (Low-Rank Adaptation):** Adds small trainable low-rank matrices to the attention layers, in order to fine-tune it with a very small subset of the original parameters,
- **IA3 (Invertible Adapter 3):** Applies element-wise learned scaling factors to attention and feedforward layers to efficiently adapt the model without touching the main weights,
- **INT8 Quantization:** Reduces the precision of feedforward layer weights from 32 bit floats to 8 bit integers, which compresses memory usage and speeding up inference.

We tested different combinations of these techniques, including using each method individually as well as combining LoRA with INT8 quantization. The models are evaluated across several metrics to study the trade-offs between efficiency and performance:

- **Memory usage** (MB)
- **Training time per epoch** (seconds)
- **Inference speed** (samples per second)
- **Test set accuracy**

# 3 Experiments and results

For these experiments, we will benchmark for these 5 models:

- Base model
- Base + LoRA adapter
- Base + $IA^3$ adapter

- INT8 Quantized model
- Combination (QLoRA: LoRA + INT8)

## 3.1 Memory usage

We first measured the memory required for the models to evaluate the effect of parameter-efficient adaptations and quantization. Table 1 summarizes the results.

| Model | Memory Usage (MB) | Memory Change (%) |
|---|---|---|
| Base | 267.82 | – |
| Base + LoRA | 270.78 | +1.1% |
| Base + IA³ | 270.34 | +0.9% |
| INT8 | 182.89 | -31.7% |
| QLoRA | 185.85 | -30.6% |

Table 1: Memory usage of DistilBERT under different parameter efficient techniques

As we expected, LoRA and IA³ add only a small number of trainable parameters, resulting in negligible increases in memory usage compared to the base model. INT8 quantization significantly reduces memory requirements by approximately 30%, while the combination of LoRA with INT8 (QLoRA) achieves similar compression with the added benefit of trainable adapters for efficient fine-tuning.

## 3.2 Training Time

We measured the training time for one epoch for each model using the same training configuration and batch size. For parameter efficient methods, only adapter parameters were updated, while the base model remained frozen. Results are reported in Table 2.

| Model | Training Time for 1 Epoch (s) |
|---|---|
| Base | 196.75 |
| Base + LoRA | 150.03 |
| Base + IA³ | 155.14 |
| INT8 | 161.08 |
| QLoRA | 148.46 |

Table 2: Training time per epoch for different techniques

LoRA and IA³ greatly reduce training time compared to full fine-tuning, by 45 and 40 seconds respectively, by only updating a small set of parameters. INT8 quantization also reduces training time, primarily due to lower memory usage during the forward pass, although it remains slower than adapter-based methods. QLoRA achieves the fastest training time by combining the quantized weights with lightweight trainable adapters.

## 3.3 Inference Speed

We evaluated inference speed by measuring the number of samples processed per second on the test set using a fixed batch size. All models were evaluated in evaluation mode with gradient computation disabled. Results are shown in Table 3.

Despite all models being evaluated in inference mode, the base model remains the slowest. This behavior is explained by the fact that `model.eval()` only disables random layers such as dropout, but does not freeze model parameters. On the

| Model | Inference Speed (samples/sec) |
|---|---|
| Base | 1489.32 |
| Base + LoRA | 3705.03 |
| Base + IA³ | 3680.59 |
| INT8 | 4810.96 |
| QLoRA | 3506.95 |

Table 3: Inference speed comparison for different techniques

other hand, PEFT methods such as LoRA and IA³ explicitly freeze the backbone weights in the backend, which leaves only a small number of parameters active (the adapter). This results in a simpler execution graph with reduced processing during the forward pass, leading to faster inference (despite technically having more weights). QLoRA is slightly slower than the two previous ones, most likely due to the fact that it requires a bit more work in the backend due to live dequantization required, combined with the extra added weights. Finally, quantization achieves the best inference performance by simply reducing memory bandwidth.

### 3.4 Test Accuracy

Table 4: Test set accuracy for different efficiency techniques

| Model | Accuracy |
|---|---|
| Base | 0.896 |
| LoRA | 0.896 |
| IA³ | 0.896 |
| INT8 Quantized | 0.900 |
| QLoRA | 0.900 |

The evaluation results show that parameter-efficient fine-tuning methods such as LoRA and IA³ achieve essentially identical accuracy to the base model, despite modifying only a small fraction of the weights. Quantization alone slightly improves accuracy, likely due to minor regularization effects from reduced precision, while QLoRA combines both techniques without loss of performance, same as quantization. Overall, PEFT methods maintain model performance while drastically reducing trainable parameters and training overhead. It is also possible that some rounding of the results happen in the backend in the *Trainer*, which makes it harder to detect small performance changes.

## 4 Conclusion

In this study, we observed that LoRA and IA³ adapters dramatically reduced the number of trainable parameters and training time without any loss in accuracy for fine-tuning, while INT8 quantization substantially decreased memory usage and improved inference speed. The combination of LoRA and quantization (QLoRA) combines these benefits. It is worth noting that this task is relatively easy for DistilBERT, which makes it challenging to observe subtle differences between the methods in terms of accuracy. Nonetheless, our experiments highlight the real-world trade-offs in memory, training efficiency, inference speed, and parameter efficiency, demonstrating the practical value of PEFT and quantization techniques.